



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Evolución de un editor
Entidad-Relación con React y
mxGraph**



Presentado por Steven Paul Alba Alba
en Universidad de Burgos — 30 de junio
de 2026

Tutor: Jesús Manuel Maudes Raedo



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Jesús Manuel Maudes Raedo, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que el alumno D. Steven Paul Alba Alba, con DNI 71755156D, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Evolución de un editor Entidad-Relación con React y mxGraph.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 30 de junio de 2026

Vº. Bº. del Tutor:

D. Jesús Manuel Maudes Raedo

Resumen

Este Trabajo Fin de Grado aborda la evolución de UBU E-R App, un editor web de diagramas Entidad-Relación desarrollado con React y mxGraph. El proyecto parte de una aplicación heredada y se centra en analizar su funcionamiento interno, estabilizar su núcleo funcional y ampliar de forma controlada los elementos del modelo Entidad-Relación soportados por la herramienta.

El trabajo se ha desarrollado con un enfoque incremental, prestando especial atención a la coherencia entre la representación gráfica del diagrama, el modelo interno mantenido por la aplicación, la validación semántica, la transformación al modelo relacional y la generación de código SQL. Entre las ampliaciones realizadas se incluyen entidades débiles, atributos compuestos y multivaluados, relaciones ternarias y un soporte inicial para jerarquías ISA. Además, se han incorporado mejoras de usabilidad, validación, importación, exportación y apoyo a la edición. El resultado debe entenderse como una evolución académica de la herramienta, orientada a mantener una traducción razonable entre el diagrama diseñado por el usuario y la salida relacional o SQL generada.

Descriptores

Modelo Entidad-Relación, diagramas E/R, React, mxGraph, transformación relacional, SQL, validación de modelos, aplicación web, jerarquías ISA

Abstract

This Bachelor's Thesis addresses the evolution of UBU E-R App, a web-based Entity-Relationship diagram editor developed with React and mxGraph. The project starts from an existing application and focuses on analysing its internal structure, stabilising its core behaviour and extending the support for several elements of the Entity-Relationship model.

The work has been carried out incrementally, paying special attention to the consistency between the visual representation of the diagram, the internal model maintained by the application, semantic validation, transformation into the relational model and SQL generation. The implemented extensions include support for weak entities, composite and multivalued attributes, ternary relationships and an initial support for ISA hierarchies. In addition, the project includes usability, validation, import, export and editing-support improvements. The resulting application should be understood as an academic evolution of the tool, aimed at maintaining a reasonable translation from user-designed Entity-Relationship diagrams to relational or SQL output.

Keywords

Entity-Relationship model, E/R diagrams, React, mxGraph, relational transformation, SQL, model validation, web application, ISA hierarchies

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Contexto del proyecto	1
1.2. Motivación	2
1.3. Punto de partida	3
1.4. Problema abordado	4
1.5. Alcance del trabajo	5
1.6. Materiales entregados	6
2. Objetivos del proyecto	7
2.1. Objetivos generales	7
2.2. Objetivos específicos	7
3. Conceptos teóricos	9
3.1. Modelo Entidad-Relación	9
3.2. Elementos básicos del modelo	10
3.3. Elementos avanzados considerados	10
3.4. Transformación al modelo relacional	14
3.5. Generación de SQL	15
3.6. Alcance teórico aplicado en el proyecto	16
4. Técnicas y herramientas	19

4.1. Entorno de desarrollo web	19
4.2. Librerías principales del editor	19
4.3. Validación, pruebas y calidad del código	20
4.4. Control de versiones, integración y despliegue	20
4.5. Documentación y redacción de la memoria	21
4.6. Sumario	21
5. Aspectos relevantes del desarrollo del proyecto	23
5.1. Planteamiento y organización del desarrollo	23
5.2. Análisis y estabilización del sistema heredado	24
5.3. Evolución del modelo interno y persistencia	24
5.4. Ampliación de los elementos Entidad-Relación soportados	27
5.5. Validación, transformación relacional y generación de SQL	27
5.6. Refinamientos de usabilidad del editor	29
5.7. Pruebas y revisión manual	30
5.8. Dificultades y decisiones técnicas relevantes	30
6. Trabajos relacionados	33
6.1. Introducción	33
6.2. Herramientas de diagramación general	33
6.3. Herramientas profesionales de modelado	34
6.4. ERDPlus	34
6.5. Comparativa final	35
7. Conclusiones y líneas de trabajo futuras	37
7.1. Conclusiones	37
7.2. Líneas de trabajo futuras	39
Bibliografía	41

Índice de figuras

1.1. Vista general del editor web con un diagrama Entidad-Relación sencillo.	2
1.2. Vista de la versión previa de Draw E-R App desplegada en Vercel.	4
1.3. Vista de la versión evolucionada de UBU E-R App desplegada en Vercel.	4
3.1. Ejemplo básico de diagrama Entidad-Relación con entidades, atributos, claves, una interrelación y cardinalidades.	9
3.2. Ejemplo de entidad débil asociada a una entidad propietaria mediante una relación identificadora.	11
3.3. Ejemplo de atributo compuesto y atributo multivaluado en una entidad.	12
3.4. Ejemplo de interrelación ternaria con tres entidades participantes y cardinalidades.	12
3.5. Ejemplo de jerarquía ISA con una entidad general y dos especializaciones.	13
5.1. Relación entre el lienzo del editor, el modelo interno y las operaciones principales de la aplicación.	26
5.2. Diálogo de validación con errores agrupados del diagrama.	28
5.3. Ejemplo de script SQL generado a partir de un diagrama validado.	29

Índice de tablas

4.1. Resumen de herramientas utilizadas en el proyecto.	21
5.1. Comparación entre la versión heredada y la versión desarrollada en este TFG.	25
6.1. Comparación entre ERDPlus y UBU E-R App.	36

1. Introducción

1.1. Contexto del proyecto

Cuando se diseña una base de datos, antes de decidir su implementación en un modelo lógico concreto, es necesario representar de forma conceptual la información del dominio y las relaciones existentes entre sus elementos. El modelo Entidad-Relación, propuesto por Chen [3], permite realizar esta representación mediante entidades, atributos e interrelaciones, facilitando el análisis del problema antes de abordar su transformación posterior.

Este Trabajo Fin de Grado se sitúa en ese contexto, pero no aborda el desarrollo de una herramienta desde cero. El proyecto se centra en la evolución de UBU E-R App, anteriormente denominada Draw E-R App, una aplicación web para la creación de diagramas Entidad-Relación desarrollada con React y mxGraph a partir del trabajo previo de Rubén Maté Iturriaga [16] y del repositorio original de la aplicación [15]. La aplicación heredada ya ofrecía una base funcional, pero requería análisis, estabilización y ampliación para mejorar la coherencia entre el diagrama editado en el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación de SQL.

A fin de ofrecer una primera visión de la aplicación, la Figura 1.1 muestra una vista general del editor, donde se aprecia el lienzo de trabajo, el panel de acciones y la representación visual de un diagrama Entidad-Relación sencillo.

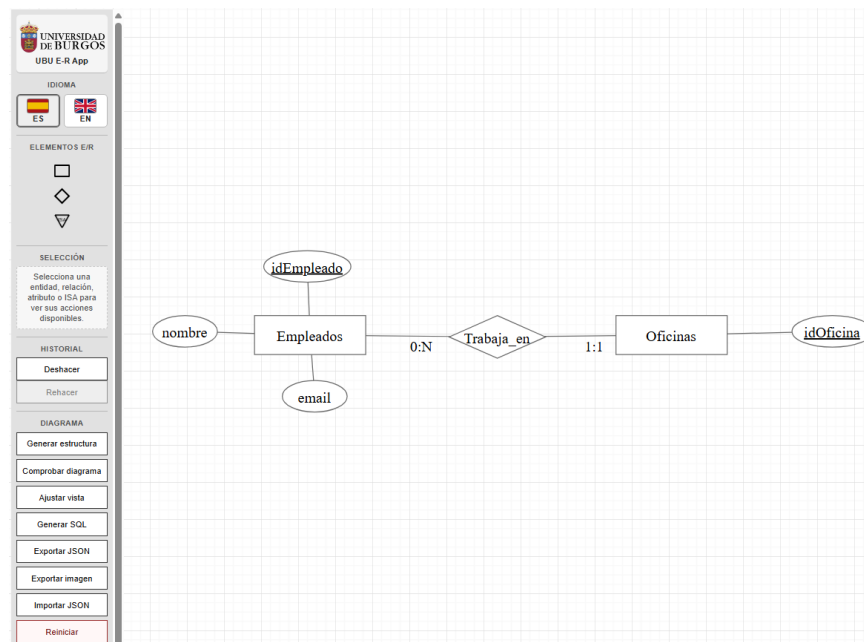


Figura 1.1: Vista general del editor web con un diagrama Entidad-Relación sencillo.

1.2. Motivación

La motivación de este trabajo surge de una necesidad práctica detectada en el aprendizaje del diseño de bases de datos. Aunque existen numerosas herramientas para crear diagramas, muchas están orientadas al modelado con UML, a diagramas físicos de bases de datos o a notaciones de tipo *crow's foot*. Algunas incluso se presentan como herramientas para diagramas E/R, aunque no sigan de forma fiel la notación utilizada habitualmente en la docencia del modelo Entidad-Relación.

Esta situación dificulta que los estudiantes puedan practicar con una herramienta gratuita, accesible y cercana a los ejemplos vistos en clase. En este contexto, resulta útil disponer de una aplicación que no solo permita dibujar entidades y relaciones, sino también validar el diagrama, entender su transformación al modelo relacional y observar el SQL derivado.

Por ello, la evolución de UBU E-R App tiene una motivación principalmente docente. El objetivo no es competir con herramientas profesionales de diseño de bases de datos, sino avanzar hacia una aplicación que ayude a practicar el modelado conceptual y refuerce la comprensión de las reglas de transformación al modelo relacional.

1.3. Punto de partida

El proyecto parte de una aplicación web previa, denominada Draw E-R App, orientada al modelado de diagramas Entidad-Relación con React y mxGraph. Esta versión ya ofrecía una base funcional para trabajar con entidades, atributos y relaciones, además de incluir mecanismos iniciales de validación y generación de SQL.

A partir de ese punto, este Trabajo Fin de Grado se centra en analizar el funcionamiento interno del editor, detectar limitaciones y evolucionarlo de forma incremental. El trabajo realizado no consistió en construir la herramienta desde cero, sino en mejorar la coherencia entre la representación visual del diagrama, el modelo interno, la validación, la transformación relacional, la generación SQL, la persistencia y la experiencia de edición.

Como estimación razonada, puede considerarse que aproximadamente un 40 % del resultado final corresponde a la base heredada y un 60 % al trabajo realizado durante este proyecto. Esta estimación no mide líneas de código, sino alcance funcional y técnico: la versión previa aportaba el editor inicial y una base de validación y generación SQL, mientras que este TFG incorpora estabilización, ampliación del soporte E/R, mejoras de transformación relacional y SQL, persistencia, usabilidad, pruebas y documentación.

La versión previa del proyecto puede consultarse en [el despliegue original en Vercel](#), mientras que la versión evolucionada está disponible en [el despliegue actual de UBU E-R App](#). Las Figuras 1.2 y 1.3 muestran la diferencia visual entre ambos despliegues: la versión heredada presenta una interfaz más básica, mientras que la versión actual incorpora una organización más completa del panel lateral, branding UBU, selector de idioma, acciones de validación, generación, importación y exportación, e información de apoyo.

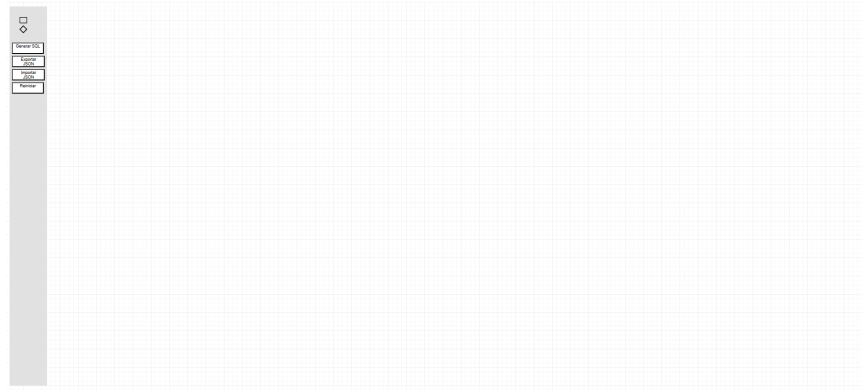


Figura 1.2: Vista de la versión previa de Draw E-R App desplegada en Vercel.

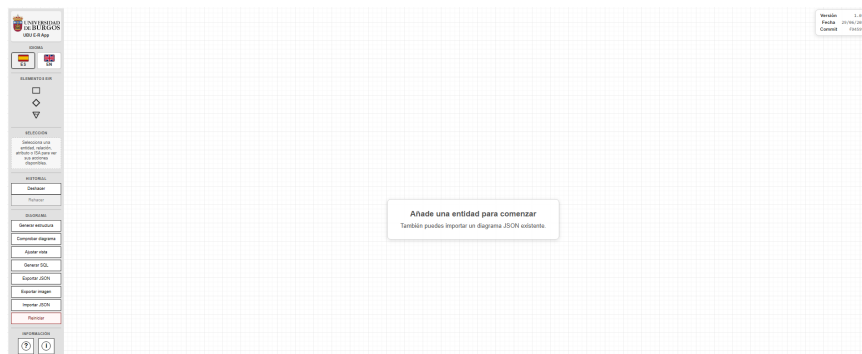


Figura 1.3: Vista de la versión evolucionada de UBU E-R App desplegada en Vercel.

1.4. Problema abordado

El principal problema abordado en este trabajo es la evolución de un editor de diagramas Entidad-Relación ya existente. En una herramienta de este tipo no basta con representar elementos en pantalla, ya que cada entidad, atributo o relación debe tener también un significado dentro del modelo interno de la aplicación.

Por este motivo, cualquier ampliación del editor debe tener en cuenta varios aspectos al mismo tiempo: el lienzo visual, los datos que se almacenan internamente, las reglas de validación, la transformación posterior al modelo relacional y la generación del script SQL. El trabajo se centra en avanzar en

esa evolución de forma controlada, revisando el comportamiento del sistema y evitando separar la parte gráfica de la parte conceptual.

1.5. Alcance del trabajo

El alcance de este Trabajo Fin de Grado se centra en la evolución de un editor web de diagramas Entidad-Relación ya existente. El proyecto no parte de una aplicación desarrollada desde cero, sino de una herramienta heredada que ya ofrecía una base funcional para la creación de diagramas, su validación y la generación de SQL.

A partir de ese punto, el trabajo realizado incluye el análisis del sistema previo, la estabilización de partes relevantes del editor y la ampliación controlada de sus capacidades. El eje principal del desarrollo ha sido mantener la coherencia entre lo que el usuario diseña en el lienzo, la representación interna del diagrama, las validaciones aplicadas, la transformación al modelo relacional y la generación de SQL.

Dentro del alcance implementado se incluyen mejoras en la representación interna del diagrama, en las reglas de validación, en la transformación al modelo relacional y en la generación del script SQL, así como la incorporación de nuevos elementos del modelo Entidad-Relación soportados por la aplicación. También se han añadido mejoras de interacción y usabilidad orientadas a facilitar el trabajo con el editor, como la selección múltiple, el acceso a información de ayuda, opciones de importación y exportación, y ajustes visuales de la interfaz.

Al tratarse de la continuación de una aplicación existente, la contribución propia de este trabajo se delimita en el análisis del sistema heredado, la estabilización de su núcleo funcional, la ampliación de elementos E/R soportados, la revisión de la transformación relacional y SQL, las mejoras de persistencia, importación, exportación y edición, así como la documentación técnica y de usuario asociada. La base inicial del editor no se presenta como desarrollo propio completo, sino como punto de partida sobre el que se ha realizado la evolución descrita.

No se pretende construir una herramienta profesional completa de diseño de bases de datos ni cubrir todas las variantes posibles del modelo Entidad-Relación. Quedan fuera del alcance implementado funcionalidades avanzadas como las agregaciones, las restricciones completas de jerarquías ISA, la herencia múltiple, la gestión de usuarios, la persistencia remota o la configuración detallada de tipos de datos SQL. Estas limitaciones se

consideran decisiones de alcance del proyecto y posibles líneas de trabajo futuro.

1.6. Materiales entregados

Junto con esta memoria se entregan los materiales necesarios para revisar, ejecutar y evaluar el proyecto:

- **Memoria y anexos:** documentación principal del TFG, incluyendo planificación, requisitos, diseño, documentación técnica, manual de usuario y sostenibilización curricular.
- **Aplicación web desplegada:** versión funcional de UBU E-R App publicada en Vercel. Acceso: [aplicación desplegada en Vercel](#).
- **Repositorio Git del proyecto:** código fuente, historial de desarrollo e instrucciones de instalación y ejecución [1]. Acceso: [repositorio del proyecto](#).
- **Material complementario:** información de ayuda disponible desde la interfaz y vídeos de demostración del funcionamiento de la aplicación.

2. Objetivos del proyecto

2.1. Objetivos generales

El objetivo principal de este Trabajo Fin de Grado es analizar y continuar el desarrollo de una aplicación web existente para el diseño de diagramas Entidad-Relación. El proyecto parte de un editor heredado que ya contaba con una base funcional, y se centra en evolucionarlo de forma controlada, ampliando sus capacidades y reforzando su coherencia interna.

Además de esta dimensión técnica, el trabajo tiene una orientación docente. La aplicación se plantea como una herramienta de apoyo para que los estudiantes puedan practicar el modelado Entidad-Relación con una notación cercana a la utilizada en la asignatura de Bases de Datos, validar sus diagramas y observar la transformación hacia el modelo relacional y SQL.

Por tanto, el objetivo general no se limita a añadir nuevas opciones al editor, sino a hacer que estas encajen dentro del funcionamiento global de la herramienta. Para ello resulta necesario mantener la relación entre el lienzo visual, el modelo interno, las reglas de validación, la transformación relacional, la generación SQL y la experiencia de edición.

2.2. Objetivos específicos

A partir del objetivo general anterior, se plantean los siguientes objetivos específicos:

- Analizar el funcionamiento del editor heredado, identificando su estructura principal, las funcionalidades existentes y las decisiones de diseño que condicionan su evolución.
- Revisar y estabilizar la relación entre los elementos visuales del lienzo y la representación interna del diagrama, evitando inconsistencias entre lo que el usuario ve y lo que la aplicación procesa.
- Ampliar el soporte del modelo Entidad-Relación incorporando elementos como entidades débiles, dependencias por identificación, atributos compuestos y multivaluados, relaciones ternarias con roles y un soporte inicial para jerarquías ISA.
- Mantener la coherencia entre la parte visual del editor, las reglas de validación, la transformación al modelo relacional, la generación de SQL y los mecanismos de persistencia, importación y exportación.
- Mejorar la validación del diagrama y la comunicación de errores al usuario, haciendo que los mensajes sean más claros y, cuando sea posible, vinculados a los elementos concretos relacionados con cada problema.
- Reforzar el uso docente de la aplicación, facilitando que el usuario pueda relacionar el diagrama Entidad-Relación que construye con las validaciones aplicadas, el modelo relacional resultante y el SQL generado.
- Incorporar mejoras de usabilidad y apoyo a la edición, como una organización más clara de la interfaz, acciones contextuales, selección múltiple, acceso a información de ayuda, soporte básico de idioma, importación, exportación y generación de estructuras de ejemplo.
- Comprobar el comportamiento de la aplicación mediante pruebas automatizadas cuando proceda y mediante revisión manual en aquellos casos que requieran validación funcional o visual.
- Documentar el trabajo realizado desde el punto de vista del desarrollo, el diseño, las pruebas, el uso de la aplicación y la evolución seguida durante el proyecto.

3. Conceptos teóricos

3.1. Modelo Entidad-Relación

El modelo Entidad-Relación es un modelo conceptual utilizado para representar la información de un dominio antes de decidir su implementación en un modelo lógico concreto. Fue propuesto por Chen como una forma de describir la información mediante entidades e interrelaciones, separando el diseño conceptual de los detalles propios de la implementación final [3].

Esta separación permite razonar primero sobre el problema que se quiere representar, sin empezar directamente por el modelo lógico, que habitualmente para los estudiantes consiste en definir tablas, columnas o sentencias SQL. Aunque en este trabajo la transformación posterior se realiza hacia el modelo relacional, el modelo E/R no depende necesariamente de dicho modelo y podría servir como punto de partida para otras formas de representación lógica.

En el contexto de este trabajo, el modelo Entidad-Relación sirve como base teórica para el editor desarrollado. La Figura 3.1 muestra un ejemplo sencillo con entidades, atributos, claves, una interrelación y cardinalidades.

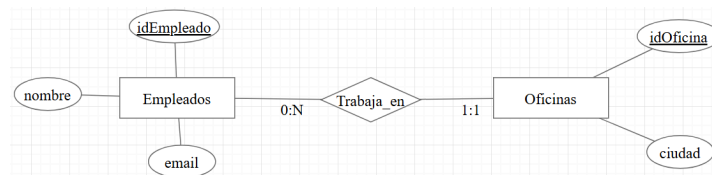


Figura 3.1: Ejemplo básico de diagrama Entidad-Relación con entidades, atributos, claves, una interrelación y cardinalidades.

3.2. Elementos básicos del modelo

Un diagrama Entidad-Relación se construye a partir de entidades, atributos, interrelaciones, cardinalidades y, en algunos casos, roles. Estos conceptos permiten describir la información de un dominio de forma visual y sirven como base para las transformaciones posteriores [17].

Entidades y atributos

Las entidades representan objetos o conceptos del dominio sobre los que se quiere almacenar información. Cada entidad se describe mediante atributos, que representan propiedades de dicha entidad o de una interrelación.

Algunos atributos actúan como identificadores, mientras que otros aportan información adicional. En el paso al modelo relacional, los atributos identificadores suelen utilizarse para definir claves primarias, ya que permiten distinguir de forma única las ocurrencias de una entidad. En la Figura 3.1, los atributos *idEmpleado* e *idOficina* aparecen subrayados porque identifican sus respectivas entidades. En cambio, atributos como *nombre*, *email* o *ciudad* describen información adicional.

Interrelaciones, cardinalidades y roles

Las interrelaciones representan asociaciones entre entidades. Las cardinalidades indican la forma en la que las entidades participan en una interrelación y condicionan su transformación posterior al modelo relacional. No se representa igual una relación uno a muchos que una relación muchos a muchos, ya que la estructura relacional resultante puede ser distinta.

En algunos casos, una misma entidad puede participar más de una vez en una interrelación. Para evitar ambigüedades se utilizan roles, que indican el papel que desempeña cada participación. Esto es especialmente útil en relaciones reflexivas o en relaciones de mayor grado con participantes repetidos.

3.3. Elementos avanzados considerados

Además de los elementos básicos, el modelo Entidad-Relación permite representar situaciones más complejas. En este trabajo no se pretende cubrir todas las variantes posibles del modelo, pero sí se han tenido en cuenta algunos elementos avanzados que resultan importantes para el editor desarrollado.

Entidades débiles

Una entidad débil es una entidad que no puede identificarse completamente por sí misma y depende de otra entidad para formar su identificación. Esta dependencia suele representarse mediante una relación identificadora entre la entidad débil y la entidad propietaria.

En estos casos, la clave de la entidad débil se construye normalmente a partir de la clave de la entidad fuerte y de un atributo propio que actúa como discriminante. Por este motivo, las entidades débiles requieren un tratamiento específico en la validación y en la transformación al modelo relacional [17].

La Figura 3.2 resume este patrón.

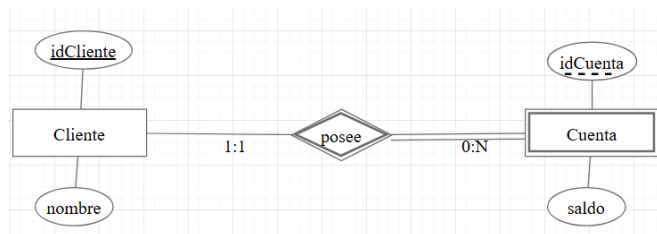


Figura 3.2: Ejemplo de entidad débil asociada a una entidad propietaria mediante una relación identificadora.

Atributos compuestos y multivaluados

Los atributos compuestos permiten descomponer una propiedad en partes más simples. Por ejemplo, un atributo *nombreCompleto* puede dividirse en nombre y apellidos. Los atributos multivaluados representan propiedades que pueden tener varios valores para una misma ocurrencia, como varios teléfonos asociados a una persona.

Estos atributos afectan a la transformación relacional. Los compuestos se descomponen en sus partes simples, mientras que los multivaluados suelen requerir una tabla propia para representar los distintos valores asociados a la entidad.

La Figura 3.3 muestra ambos casos.

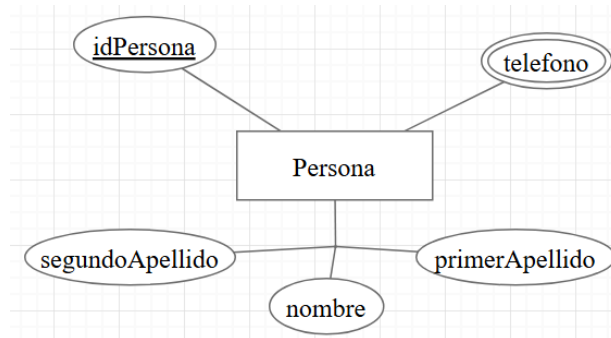


Figura 3.3: Ejemplo de atributo compuesto y atributo multivaluado en una entidad.

Relaciones de grado superior y reflexivas

Además de las relaciones binarias, el modelo E/R permite representar interrelaciones en las que participan más de dos entidades. Las relaciones ternarias son especialmente relevantes porque no siempre pueden sustituirse por varias relaciones binarias sin perder significado. En ellas, las cardinalidades deben interpretarse teniendo en cuenta la combinación de los otros participantes.

También pueden existir relaciones reflexivas, en las que una entidad se relaciona consigo misma. En estos casos los roles permiten distinguir las distintas participaciones de la misma entidad.

La Figura 3.4 muestra un ejemplo de relación ternaria.

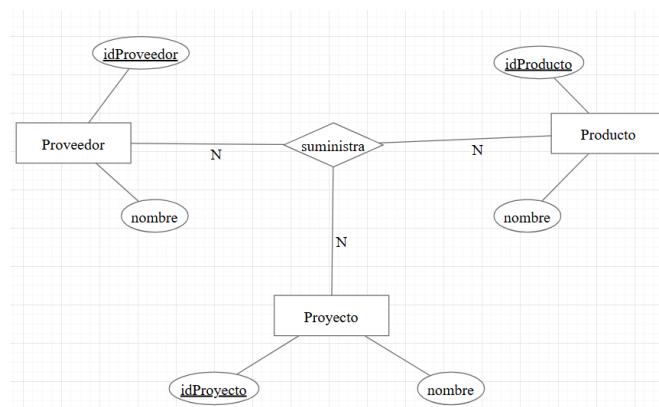


Figura 3.4: Ejemplo de interrelación ternaria con tres entidades participantes y cardinalidades.

Jerarquías ISA

Las jerarquías ISA permiten representar especialización y generalización entre entidades. Una entidad general recoge atributos comunes y las especializaciones representan subconjuntos con características propias. Según la teoría del modelo E/R, estas jerarquías pueden incluir restricciones de totalidad, parcialidad, solapamiento, exclusividad o discriminantes [17].

En este trabajo se considera un soporte inicial: una entidad general, una o varias especializaciones y herencia de clave. La Figura 3.5 muestra un ejemplo de jerarquía ISA sencilla.

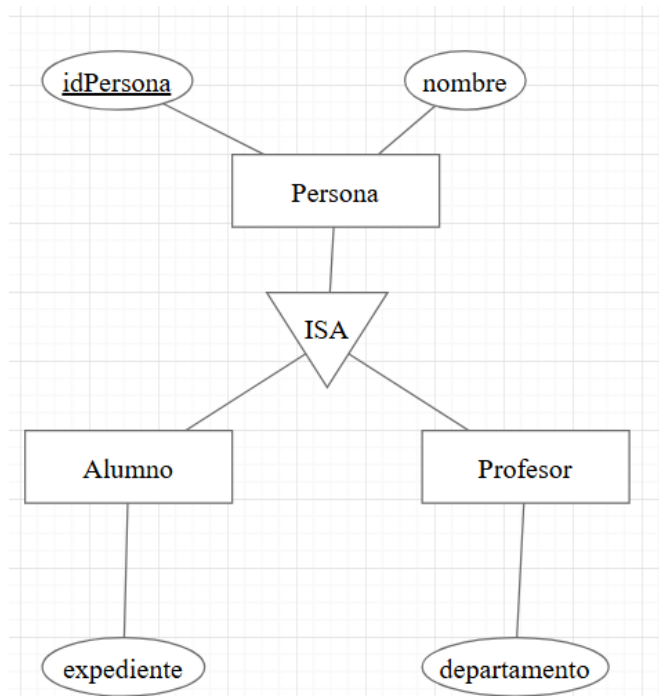


Figura 3.5: Ejemplo de jerarquía ISA con una entidad general y dos especializaciones.

Agregaciones

Las agregaciones permiten tratar una interrelación como un elemento de nivel superior dentro del modelo Entidad-Relación, de forma que pueda participar en otra relación con nuevos elementos del dominio [17]. Su incorporación a un editor visual requiere decisiones específicas sobre representación gráfica, validación, persistencia y transformación al modelo relacional.

En este trabajo las agregaciones se consideran únicamente como parte del contexto teórico del modelo, pero no forman parte del alcance implementado. Su soporte se deja como una posible línea de trabajo futura.

3.4. Transformación al modelo relacional

El modelo Entidad-Relación describe un dominio desde un punto de vista conceptual. Para implementarlo en una base de datos relacional es necesario transformarlo en relaciones, atributos, claves primarias y claves foráneas. Esta transformación actúa como paso intermedio entre el diagrama E/R y la generación final de SQL.

Conviene distinguir esta fase de la generación de SQL. El modelo relacional puede representarse de forma abstracta, indicando tablas, columnas, claves primarias y claves foráneas, sin escribir todavía sentencias `CREATE TABLE`. Por ejemplo, el diagrama de la Figura 3.1 podría expresarse como `Oficinas(idOficina PK, ciudad)` y `Empleados(idEmpleado PK, nombre, email, idOficina FK)`.

De forma general, las entidades se transforman en tablas, sus atributos en columnas y sus identificadores en claves primarias. Las interrelaciones condicionan la aparición de claves foráneas o tablas intermedias según su cardinalidad.

En las relaciones uno a muchos, la clave primaria de la entidad situada en el lado uno se incorpora como clave foránea en la tabla del lado muchos. En las relaciones uno a uno también se utiliza una clave foránea, pero esta debe declararse como única mediante una restricción `UNIQUE`, salvo que se integre directamente en la clave primaria.

En las relaciones muchos a muchos se crea una tabla intermedia formada por las claves de las entidades participantes. Normalmente, la clave primaria de esta tabla es compuesta y está formada por esas claves foráneas. Si la relación tiene atributos propios, estos se añaden como columnas de la tabla intermedia.

Las relaciones ternarias se transforman mediante una tabla propia con claves foráneas hacia las tres entidades participantes. La clave primaria depende de las cardinalidades: en el caso más general puede estar formada por las tres claves, mientras que si existe una participación con cardinalidad máxima uno pueden aparecer claves candidatas más reducidas formadas por los otros participantes.

Otros elementos requieren reglas específicas. Los atributos multivaluados suelen generar tablas propias; las entidades débiles incorporan la clave de su entidad propietaria como parte de su identificación; y las relaciones identificadoras condicionan tanto claves primarias como claves foráneas.

En el caso de las jerarquías ISA, existen varias estrategias de transformación. En este trabajo se adopta una solución sencilla: la entidad general se transforma en una tabla ordinaria y cada especialización en una tabla propia que incorpora la clave de la generalización como clave primaria y clave foránea [17].

Esta fase es especialmente importante en el proyecto porque conecta el diagrama visual diseñado por el usuario con la salida SQL generada por la aplicación.

3.5. Generación de SQL

SQL es el lenguaje utilizado habitualmente para definir y manipular bases de datos relacionales. En este trabajo interesa especialmente la parte relacionada con la creación de tablas, claves primarias, claves foráneas y restricciones de unicidad. El SQL generado utiliza una sintaxis sencilla, basada en construcciones habituales del estándar ANSI/ISO SQL y compatible con PostgreSQL [25].

Una vez transformado el diagrama E/R en un modelo relacional, la aplicación genera sentencias `CREATE TABLE`. Las tablas se ordenan para que muchas claves foráneas puedan declararse directamente dentro de la tabla correspondiente, evitando recurrir a `ALTER TABLE` cuando no es necesario.

Por ejemplo, una relación uno a muchos entre oficinas y empleados podría generarse separando la clave foránea:

```
CREATE TABLE Oficinas (  
  idOficina VARCHAR(40),  
  ciudad VARCHAR(40),  
  PRIMARY KEY (idOficina)  
);
```

```
CREATE TABLE Empleados (  
  idEmpleado VARCHAR(40),  
  nombre VARCHAR(40),  
  email VARCHAR(40),
```

```
    idOficina VARCHAR(40),  
    PRIMARY KEY (idEmpleado)  
);  
  
ALTER TABLE Empleados  
ADD FOREIGN KEY (idOficina)  
REFERENCES Oficinas(idOficina);
```

Si la tabla referenciada se crea antes, la clave foránea puede incluirse directamente en la definición de `Empleados`:

```
CREATE TABLE Empleados (  
    idEmpleado VARCHAR(40),  
    nombre VARCHAR(40),  
    email VARCHAR(40),  
    idOficina VARCHAR(40),  
    PRIMARY KEY (idEmpleado),  
    FOREIGN KEY (idOficina)  
        REFERENCES Oficinas  
);
```

Este segundo enfoque resulta más legible en esquemas con varias tablas, porque las columnas y sus restricciones aparecen juntas. Las sentencias `ALTER TABLE` se mantienen como mecanismo de respaldo cuando existen dependencias circulares que no pueden resolverse únicamente mediante la ordenación.

El SQL generado debe entenderse como una traducción académica y simplificada del modelo diseñado. Por ejemplo, los atributos se traducen con un tipo general `VARCHAR(40)`, suficiente para representar la estructura del esquema, pero sin entrar en una configuración detallada de tipos físicos, índices u optimizaciones propias de una base de datos real.

3.6. Alcance teórico aplicado en el proyecto

El modelo Entidad-Relación incluye más conceptos de los que se implementan en este trabajo. Por ello, el alcance aplicado se centra en aquellos elementos que se han incorporado realmente al editor: entidades fuertes y débiles, atributos simples, compuestos y multivaluados, relaciones binarias,

reflexivas y ternarias, cardinalidades, roles, relaciones identificadoras y un soporte inicial para jerarquías ISA.

En el caso de ISA, el soporte se limita a una generalización con una o varias especializaciones y herencia de clave hacia las tablas especializadas. No se incluyen restricciones más avanzadas como totalidad o parcialidad, especializaciones solapadas o exclusivas, discriminantes ni herencia múltiple. Las agregaciones también quedan fuera del alcance implementado.

Este alcance permite trabajar con varios elementos relevantes del modelo E/R sin convertir la herramienta en un editor completo de todas las variantes teóricas posibles.

4. Técnicas y herramientas

4.1. Entorno de desarrollo web

El proyecto se desarrolla como una aplicación web ejecutada en el navegador. Este enfoque permite utilizar el editor sin instalar una aplicación de escritorio específica, manteniendo gran parte de la lógica en el cliente web.

La aplicación está desarrollada principalmente en JavaScript y utiliza Node.js [23] y npm [22] para gestionar dependencias y ejecutar los scripts del proyecto. Mediante estos scripts se arranca la aplicación en modo desarrollo, se genera la versión de producción y se lanzan las pruebas definidas.

Durante el desarrollo se utilizó Visual Studio Code [20] como editor principal. Esta herramienta permitió trabajar con el código fuente, revisar la estructura del proyecto, utilizar integración con Git y modificar tanto los ficheros de implementación como la documentación asociada.

4.2. Librerías principales del editor

La interfaz del proyecto se desarrolla con React [18], una librería de JavaScript orientada a la construcción de interfaces mediante componentes. En este trabajo, React sirve para organizar la aplicación, gestionar las partes visuales del editor y coordinar la interacción del usuario.

Para la representación y manipulación de los diagramas se utiliza mx-Graph [11]. Esta librería proporciona la base gráfica necesaria para trabajar con vértices, aristas y eventos dentro del área de edición. En mxGraph, los elementos gráficos se representan mediante celdas: una celda puede actuar

como vértice, cuando representa un elemento situado en el lienzo, o como arista, cuando representa una conexión entre elementos.

En este proyecto, entidades, atributos, relaciones, jerarquías ISA y cardinalidades se apoyan en celdas de `mxGraph` para su visualización. Estas celdas deben mantenerse sincronizadas con el modelo interno del diagrama, que es el que posteriormente se utiliza para validar, transformar y generar SQL.

Además, se utiliza Material UI [21] para algunos componentes de la interfaz, como botones, diálogos o controles de interacción, manteniendo una apariencia más uniforme dentro de la aplicación.

4.3. Validación, pruebas y calidad del código

El proyecto utiliza distintas herramientas para revisar el comportamiento de la aplicación y mantener la calidad del código. Estas herramientas no sustituyen a la revisión manual del editor, especialmente en los aspectos visuales o de interacción, pero ayudan a detectar errores y regresiones durante el desarrollo.

Para las pruebas unitarias se utiliza Vitest [29], que permite comprobar funciones concretas de validación, transformación o generación de resultados. Además, el proyecto cuenta con pruebas end-to-end mediante Playwright [19], que ejecutan escenarios completos sobre la aplicación en navegador.

Junto a las pruebas, se utiliza Biome [2] como herramienta de formateo y análisis estático. También se emplea Lefthook [14] para ejecutar comprobaciones asociadas al flujo de trabajo con Git.

4.4. Control de versiones, integración y despliegue

Para el control de versiones se utiliza Git [8], mientras que GitHub [9] se emplea como repositorio remoto y como apoyo para la organización del desarrollo mediante issues, ramas, commits y milestones.

También se utilizaron mecanismos de comprobación mediante GitHub Actions [10]. En el proyecto se definen dos flujos principales: uno orientado a calidad del código, que ejecuta Biome sobre el repositorio, y otro orientado a pruebas, que instala dependencias, prepara los navegadores de Playwright,

ejecuta las pruebas unitarias con `npm test`, lanza las pruebas end-to-end y conserva el informe de Playwright como artefacto de la ejecución.

Además del entorno local, se utilizó Vercel [26] para disponer de versiones desplegadas de la aplicación durante el desarrollo. Estos despliegues permitieron validar prototipos funcionales desde el navegador y facilitar la revisión de cambios sin depender siempre de una instalación local.

4.5. Documentación y redacción de la memoria

La memoria y los anexos se han redactado en LaTeX siguiendo la plantilla del Trabajo Fin de Grado. Para ello se utilizó Overleaf [24], que permitió editar la documentación de forma progresiva y comprobar la compilación de los documentos durante el desarrollo.

4.6. Sumario

La Tabla 4.1 resume el papel principal de las herramientas utilizadas durante el desarrollo y la documentación del proyecto.

Tabla 4.1: Resumen de herramientas utilizadas en el proyecto.

Herramienta	Uso en el proyecto
Node.js, npm y Visual Studio Code	Gestión del entorno de desarrollo, dependencias, scripts y edición del código fuente.
React, mxGraph y Material UI	Construcción de la interfaz, representación gráfica del diagrama y componentes visuales de apoyo.
Vitest y Playwright	Pruebas unitarias y pruebas end-to-end sobre la aplicación en navegador.
Biome y Lefthook	Formateo, análisis estático y comprobaciones asociadas al flujo de trabajo con Git.
Git, GitHub y GitHub Actions	Control de versiones, organización de tareas, trazabilidad y ejecución automática de comprobaciones.
Vercel	Despliegue de versiones funcionales de la aplicación para validación y revisión desde el navegador.
Overleaf y LaTeX	Redacción y compilación de la memoria y los anexos.

5. Aspectos relevantes del desarrollo del proyecto

5.1. Planteamiento y organización del desarrollo

El desarrollo de este trabajo no comenzó desde una aplicación vacía, sino a partir de un editor de diagramas Entidad-Relación ya existente. Antes de añadir nuevas funcionalidades fue necesario comprender cómo estaba construido el sistema, qué partes podían aprovecharse y qué aspectos convenía revisar para poder seguir ampliándolo con seguridad.

Una de las ideas principales del proyecto ha sido mantener la coherencia entre las distintas partes del editor. No basta con que un elemento se dibuje correctamente en el lienzo: también debe quedar representado en el modelo interno, ya que esa información se utiliza después para guardar el diagrama, reconstruirlo, validarlo, transformarlo al modelo relacional y generar SQL.

Por ello, el desarrollo se planteó de forma incremental, utilizando ramas, commits, issues y milestones de GitHub para separar tareas y mantener trazabilidad. Esta organización resultó especialmente útil al trabajar sobre una aplicación heredada, donde un cambio podía afectar simultáneamente a la interfaz, el modelo interno, la persistencia, la validación o la generación de SQL.

La Tabla 5.1 resume la diferencia entre el punto de partida heredado y la aportación realizada en este trabajo. La comparación no pretende desvalorizar la aplicación previa, sino delimitar qué partes se han reutilizado

y qué aspectos se han ampliado, estabilizado o documentado durante este TFG.

5.2. Análisis y estabilización del sistema heredado

Antes de incorporar ampliaciones, se realizó una revisión del sistema heredado. La aplicación ya permitía trabajar con diagramas E/R básicos, pero era necesario comprender la relación entre el lienzo de mxGraph, el modelo interno y las operaciones posteriores de validación y generación.

Durante este análisis se identificaron varios puntos críticos. En primer lugar, la aplicación funcionaba en dos niveles: la parte visual, gestionada mediante mxGraph, y el modelo interno, utilizado por la lógica de la aplicación. Si ambos niveles no se actualizan de forma coherente, puede ocurrir que el usuario vea un diagrama en pantalla que no coincida con la información que después se valida o se transforma a SQL.

También se revisaron los mecanismos de carga, guardado, importación y exportación. En una aplicación de este tipo, la persistencia no consiste únicamente en almacenar un dibujo, sino en conservar suficiente información semántica para reconstruir el diagrama y seguir trabajando con él. Por ello, se normalizaron estructuras internas, se revisó la reconstrucción visual y se mejoró el tratamiento de identificadores al importar o combinar diagramas.

Ejemplos concretos de estabilización fueron la revisión de la sincronización entre celdas visuales y modelo interno, la normalización de diagramas con campos incompletos, el remapeo de identificadores al importar JSON y la mejora de los mensajes de validación para indicar, cuando es posible, el elemento relacionado con cada problema.

5.3. Evolución del modelo interno y persistencia

El modelo interno del diagrama actúa como estructura central del editor. A partir de él se validan los elementos, se transforma el diagrama al modelo relacional, se genera SQL y se reconstruye el contenido al cargar un fichero o recuperar el estado guardado.

La Figura 5.1 resume esta relación entre el lienzo visual, el modelo interno y las operaciones principales de la aplicación.

Tabla 5.1: Comparación entre la versión heredada y la versión desarrollada en este TFG.

Aspecto	Versión heredada	Aportación de este TFG
Interfaz del editor	Controles básicos sobre el lienzo.	Reorganización del panel lateral, branding UBU, selector de idioma, acciones contextuales, textos de apoyo e información del proyecto.
Modelo interno	Representación inicial de entidades, atributos y relaciones.	Ampliación del metamodelo para entidades débiles, atributos compuestos y multivaluados, ternarias, roles, relaciones identificadoras e ISA inicial.
Validación	Comprobaciones básicas antes de generar resultados.	Reglas ampliadas, mensajes agrupados y localización más precisa del elemento relacionado con cada error cuando es posible.
Transformación relacional	Transformación básica hacia tablas y claves.	Revisión de casos complejos: multivaluados, entidades débiles, ternarias, roles, claves compuestas e ISA con clave heredada.
Generación SQL	Generación inicial de sentencias SQL.	SQL más legible, ordenación de tablas para reducir <code>ALTER TABLE</code> , simplificación de claves foráneas, restricciones <code>UNIQUE</code> y normalización de identificadores.
Persistencia y usabilidad	Persistencia e interacción básicas.	Importación/exportación JSON, reconstrucción visual, selección múltiple, ajuste de vista, exportación visual y estructuras de ejemplo.
Pruebas y documentación	Base inicial de desarrollo y documentación previa.	Ampliación de pruebas, revisión manual de casos complejos y documentación técnica, de usuario y de diseño actualizada.

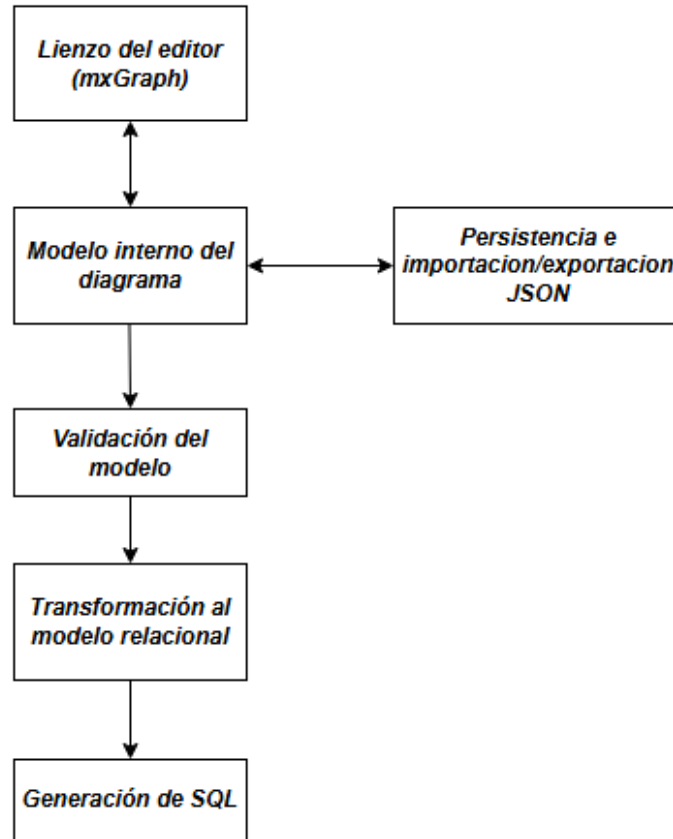


Figura 5.1: Relación entre el lienzo del editor, el modelo interno y las operaciones principales de la aplicación.

Durante el proyecto se amplió este modelo para incorporar información que antes no era necesaria o no estaba representada de forma suficiente. Esto incluye propiedades asociadas a entidades débiles, discriminantes, atributos compuestos y multivaluados, relaciones ternarias, roles, relaciones identificadoras y jerarquías ISA.

También se revisó la persistencia local y la importación/exportación JSON. La exportación permite guardar un diagrama en un formato reutilizable, mientras que la importación permite recuperar diagramas previos. Además, se incorporaron modos de importación para reemplazar el diagrama actual o combinarlo con el existente, evitando conflictos mediante el remapeo de identificadores y el desplazamiento visual del bloque importado.

5.4. Ampliación de los elementos Entidad-Relación soportados

Una parte relevante del trabajo consistió en ampliar los elementos del modelo Entidad-Relación soportados por la aplicación. Estas ampliaciones no se limitaron a añadir nuevas formas visuales, sino que exigieron adaptar el modelo interno, la validación, la persistencia, la transformación relacional y la generación SQL.

Entre los elementos incorporados o revisados se encuentran:

- **Entidades débiles y relaciones identificadoras:** se añadió soporte para entidades que dependen de una entidad fuerte y utilizan un discriminante como parte de su identificación.
- **Atributos compuestos y multivaluados:** se revisó su representación visual e interna, así como su transformación posterior al modelo relacional.
- **Relaciones ternarias y roles:** se amplió el soporte de relaciones de grado superior, incluyendo participantes repetidos mediante roles para evitar ambigüedades.
- **Jerarquías ISA:** se incorporó un soporte inicial para generalizaciones y especializaciones, con herencia de clave hacia las tablas especializadas.

En el caso de ISA, el alcance se mantuvo deliberadamente limitado. La aplicación permite una generalización y una o varias especializaciones, pero no incorpora restricciones de totalidad o parcialidad, solapamiento o exclusividad, discriminantes ni herencia múltiple. Esta decisión permitió ofrecer una primera base funcional sin aumentar excesivamente la complejidad del proyecto.

5.5. Validación, transformación relacional y generación de SQL

La validación del diagrama es una de las partes más importantes de la herramienta, ya que evita generar resultados a partir de modelos incompletos o incoherentes. Entre las comprobaciones realizadas se incluyen reglas generales, como evitar elementos sin nombre o nombres repetidos cuando pueden producir ambigüedad, y reglas específicas del modelo E/R.

Por ejemplo, se revisa que cada entidad tenga atributos y algún mecanismo de identificación válido: una clave primaria en entidades fuertes, un discriminante en entidades débiles o una clave heredada cuando la entidad actúa como especialización ISA. También se comprueba que las relaciones tengan participantes y cardinalidades configurados de forma válida.

La Figura 5.2 muestra un ejemplo de errores detectados durante la validación. En la versión heredada, la validación podía indicar el tipo de problema, pero no siempre permitía localizar claramente dónde se producía. En la versión actual, cuando la información está disponible, el mensaje añade el elemento concreto relacionado con el error, por ejemplo indicando qué entidad no tiene clave primaria.

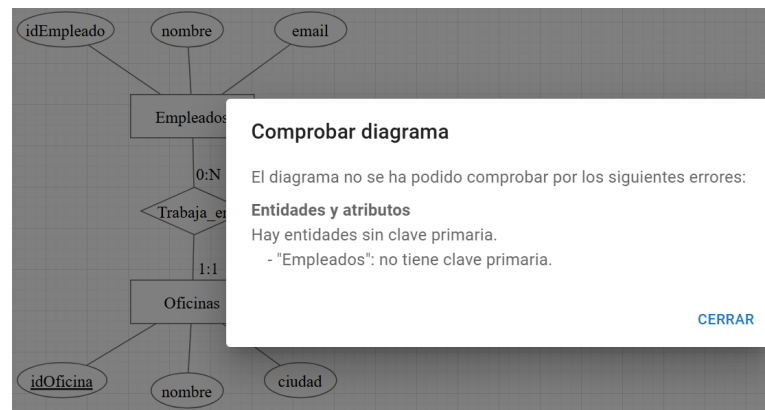


Figura 5.2: Diálogo de validación con errores agrupados del diagrama.

Tras la validación, la aplicación transforma el diagrama al modelo relacional. Esta transformación incluye la creación de tablas para entidades, relaciones muchos a muchos, atributos multivaluados, relaciones ternarias, entidades débiles y especializaciones ISA. También se revisa la propagación de claves primarias y claves foráneas según las cardinalidades y el tipo de elemento representado.

La generación de SQL se apoya en esa representación relacional. El SQL generado utiliza una sintaxis sencilla, basada en construcciones habituales del estándar ANSI/ISO SQL y compatible con PostgreSQL [25]. En particular, se emplean sentencias `CREATE TABLE`, claves primarias, claves foráneas, restricciones `UNIQUE` y restricciones `NOT NULL`.

Durante la revisión del generador se buscó producir un script más legible. Para ello, las claves foráneas se incluyen dentro de las sentencias `CREATE TABLE` cuando las dependencias entre tablas pueden resolverse mediante el

orden de creación. Las sentencias `ALTER TABLE` se mantienen como respaldo para dependencias circulares que no pueden resolverse solo mediante la ordenación.

También se simplificó la representación de las claves foráneas que referencian la clave primaria completa de la tabla destino. Por ejemplo, en lugar de generar `REFERENCES Oficinas(idOficina)`, puede generarse `REFERENCES Oficinas`. Además, se normalizan identificadores para evitar problemas con espacios, acentos u otros caracteres poco adecuados para SQL.

La Figura 5.3 muestra un ejemplo de script SQL generado por la aplicación.

```
DROP TABLE IF EXISTS Oficinas, Empleados CASCADE;

CREATE TABLE Oficinas (
  idOficina VARCHAR(40) PRIMARY KEY,
  nombre VARCHAR(40),
  ciudad VARCHAR(40)
);

CREATE TABLE Empleados (
  idEmpleado VARCHAR(40) PRIMARY KEY,
  nombre VARCHAR(40),
  email VARCHAR(40),
  idOficina_Trabaja_en VARCHAR(40) NOT NULL REFERENCES Oficinas
);
```

Figura 5.3: Ejemplo de script SQL generado a partir de un diagrama validado.

El resultado debe entenderse como una traducción académica y simplificada del modelo diseñado. Por ejemplo, los atributos se traducen con un tipo general `VARCHAR(40)`, suficiente para representar la estructura del esquema, pero sin entrar en una configuración detallada de tipos físicos.

5.6. Refinamientos de usabilidad del editor

Además de las ampliaciones del modelo, se incorporaron mejoras orientadas a facilitar el uso del editor. Entre ellas se encuentran la reorganización del panel lateral, el branding UBU, el selector de idioma, la selección múltiple visible, el borrado y movimiento conjunto de elementos, el ajuste de vista, la exportación visual, la importación JSON y la generación de estructuras de ejemplo.

También se revisaron mensajes de confirmación, validación y error para que resultaran más claros. Estas mejoras no modifican la teoría del modelo

Entidad-Relación, pero reducen fricción durante la edición y ayudan a que la herramienta resulte más usable en un contexto docente.

5.7. Pruebas y revisión manual

La comprobación del proyecto se realizó combinando pruebas automáticas y revisión manual. Las pruebas unitarias se utilizaron para validar funciones de dominio, transformación y generación, mientras que las pruebas end-to-end permitieron comprobar flujos completos de uso sobre la aplicación en navegador.

Además, se realizó revisión manual en aquellos casos donde era importante comprobar el comportamiento visual o la interacción del usuario, como la reconstrucción del lienzo, la selección múltiple, la importación de diagramas, la edición de elementos o la generación visual de estructuras.

En las comprobaciones recientes se ejecutaron los siguientes comandos:

```
npm run lint
npm test
npm run test:e2e
npm run build
```

Estas comprobaciones permitieron verificar que el proyecto mantenía un estado estable antes de la fase final de documentación y revisión.

5.8. Dificultades y decisiones técnicas relevantes

El desarrollo presentó varias dificultades derivadas del carácter heredado y visual de la aplicación. Una de las principales fue mantener la sincronización entre mxGraph y el modelo interno, ya que el lienzo representa elementos gráficos, mientras que la aplicación necesita conservar información semántica para validar, transformar y generar SQL.

Otra dificultad fue decidir el alcance de las ampliaciones. Algunos elementos, como las jerarquías ISA, podían crecer mucho en complejidad si se incorporaban todas sus variantes teóricas. Por ello, se optó por un soporte inicial controlado, dejando fuera restricciones de totalidad o parcialidad, solapamiento o exclusividad, discriminantes y herencia múltiple.

También fue necesario equilibrar la legibilidad del SQL generado con la complejidad de las dependencias entre tablas. Por esta razón se priorizó incluir claves foráneas dentro de `CREATE TABLE` cuando el orden de creación lo permite, manteniendo `ALTER TABLE` únicamente como mecanismo de respaldo para dependencias circulares.

En conjunto, estas decisiones reflejan el enfoque seguido durante el proyecto: evolucionar la aplicación de forma mantenible, ampliando su utilidad docente sin convertirla en una herramienta profesional completa de diseño de bases de datos.

6. Trabajos relacionados

6.1. Introducción

Existen numerosas herramientas para crear diagramas relacionados con bases de datos. Sin embargo, no todas persiguen el mismo objetivo ni utilizan la misma notación. Algunas están orientadas al dibujo general de diagramas, otras al modelado profesional de bases de datos y otras al apoyo docente en el aprendizaje del modelo Entidad-Relación.

La comparación realizada en este capítulo no pretende establecer una clasificación absoluta de herramientas, sino situar UBU E-R App frente a alternativas que pueden utilizarse con fines similares. Para ello se consideran criterios como la disponibilidad gratuita o de pago, la facilidad de acceso, la cercanía a la notación Entidad-Relación académica, la validación del modelo, la transformación relacional, la generación SQL y la experiencia de uso.

6.2. Herramientas de diagramación general

diagrams.net, conocido también como draw.io, es una herramienta general de diagramación que permite crear distintos tipos de diagramas, entre ellos diagramas relacionados con software, redes, UML o bases de datos [12]. Su principal ventaja es que ofrece una interfaz madura, flexible y gratuita. Sin embargo, su enfoque es el de una herramienta de dibujo general: permite representar visualmente un diagrama E/R, pero no interpreta semánticamente sus elementos, no valida reglas propias del modelo E/R y no genera una transformación relacional controlada.

Excalidraw es una herramienta de pizarra virtual colaborativa orientada a crear esquemas y diagramas con estética de dibujo manual [6]. Dispone de

una versión gratuita y de una modalidad Excalidraw+ con funcionalidades adicionales [7]. Puede resultar útil para bocetos o explicaciones informales, pero no está especializada en el modelo Entidad-Relación ni en la generación de esquemas relacionales o SQL.

Estas herramientas son útiles para dibujar o comunicar ideas, pero no cubren el objetivo principal de UBU E-R App: conectar la edición visual del diagrama con su validación, transformación relacional y generación SQL.

6.3. Herramientas profesionales de modelado

Lucidchart es una herramienta profesional de diagramación en línea que ofrece un plan gratuito y planes de pago orientados a usuarios individuales, equipos y organizaciones [13]. Su ámbito es amplio y está pensado para distintos tipos de diagramas, colaboración y documentación visual.

Visual Paradigm ofrece herramientas específicas para modelado de bases de datos y creación de diagramas ERD. Su versión online incluye una herramienta gratuita para crear diagramas ERD, mientras que otras funcionalidades forman parte de planes o productos de pago [27, 28].

Estas soluciones son más completas desde el punto de vista profesional, especialmente cuando se trabaja con modelos físicos, documentación empresarial o integración con otros procesos de diseño. No obstante, esa amplitud puede alejarlas del objetivo concreto de este TFG: proporcionar una herramienta docente, sencilla y centrada en la notación Entidad-Relación trabajada en la asignatura.

6.4. ERDPlus

ERDPlus es la alternativa más próxima al enfoque de UBU E-R App. Se trata de una herramienta web gratuita para crear diagramas Entidad-Relación, esquemas relacionales, esquemas en estrella y generar sentencias SQL [4]. Además, su propia documentación presenta el servicio como una herramienta gratuita y durante la revisión manual se observó la presencia de publicidad en la interfaz [4, 5].

Frente a herramientas de dibujo general, ERDPlus sí interpreta parte del contenido del diagrama y permite transformar modelos E/R en esquemas relacionales y SQL. Por este motivo, resulta especialmente relevante como

punto de comparación. También incorpora soporte para elementos que no están completamente implementados en UBU E-R App, como ciertas formas de especialización o agregación.

No obstante, durante la revisión manual realizada para este trabajo se observaron varias diferencias importantes. Por ejemplo, algunas operaciones de edición resultan menos directas, la gestión de especializaciones puede ser rígida y la generación SQL simplifica decisiones que en un contexto docente son relevantes, como la distinción entre cardinalidad mínima cero y uno o el tratamiento de determinadas relaciones uno a uno mediante restricciones `UNIQUE`.

La Tabla 6.1 resume las principales diferencias observadas entre ERDPlus y UBU E-R App desde el punto de vista del alcance de este trabajo.

6.5. Comparativa final

En conjunto, las alternativas revisadas permiten situar UBU E-R App en un espacio intermedio. No pretende competir con herramientas profesionales de modelado de bases de datos ni con editores gráficos de propósito general. Su aportación se encuentra en ofrecer una herramienta gratuita, accesible, sin publicidad, ejecutable localmente y orientada al aprendizaje del modelo Entidad-Relación, manteniendo conectadas la edición visual, la representación interna, la validación, la transformación relacional y la generación SQL.

La herramienta más cercana es ERDPlus, pero la comparación muestra que UBU E-R App resulta más ajustada al alcance docente de este TFG en aspectos como la ausencia de publicidad, la posibilidad de ejecución local, el soporte de relaciones ternarias con roles, la validación orientada al usuario y el control explícito de ciertos casos de transformación relacional.

Tabla 6.1: Comparación entre ERDPlus y UBU E-R App.

Aspecto	ERDPlus	UBU E-R App
Disponibilidad	Herramienta web gratuita, con almacenamiento en la nube y publicidad.	Herramienta web gratuita, con despliegue web y posibilidad de ejecución local.
Enfoque	Modelado E/R, esquemas relacionales, esquemas en estrella y generación SQL.	Modelado E/R académico, validación, transformación relacional y generación SQL simplificada.
Elementos soportados	Soporta entidades, entidades débiles, atributos, cardinalidades y algunos elementos avanzados.	Soporta entidades fuertes y débiles, atributos compuestos y multivaluados, binarias, ternarias, reflexivas, roles, identificadoras e ISA inicial.
ISA y agregaciones	Incluye soporte para especializaciones y mecanismos próximos a agregaciones o entidades asociativas.	ISA inicial con clave heredada; las agregaciones quedan fuera del alcance implementado.
Transformación SQL	Genera SQL automáticamente, aunque en la revisión manual se observaron simplificaciones discutibles en algunos casos.	Genera SQL compatible con PostgreSQL, intentando respetar cardinalidades, claves primarias, claves foráneas, UNIQUE, NOT NULL y dependencias entre tablas.
Usabilidad	Interfaz especializada, dependiente de conexión y con publicidad.	Interfaz adaptada al proyecto, sin publicidad, con persistencia local, importación/exportación JSON, selección múltiple, ajuste de vista y estructuras de ejemplo.

7. Conclusiones y líneas de trabajo futuras

En este capítulo se recogen las principales conclusiones obtenidas tras el desarrollo del proyecto y se señalan algunas posibles líneas de continuación. El objetivo no es repetir todo el trabajo realizado, sino valorar el resultado alcanzado y los aspectos que podrían ampliarse en el futuro.

7.1. Conclusiones

El desarrollo de este trabajo ha permitido evolucionar una aplicación web ya existente para el diseño de diagramas Entidad-Relación, ampliando sus capacidades y mejorando su coherencia interna. Al no partir de cero, una parte importante del trabajo consistió en comprender el sistema heredado, estabilizar puntos delicados y adaptar su estructura para incorporar nuevas funcionalidades de forma controlada.

Una de las conclusiones principales es que, en una herramienta visual de modelado, no basta con representar correctamente los elementos en pantalla. Cada elemento del diagrama debe estar reflejado también en el modelo interno de la aplicación, ya que esa información se utiliza después para guardar el diagrama, reconstruirlo, validarlo, transformarlo al modelo relacional y generar el script SQL.

A lo largo del proyecto se ha ampliado el soporte del modelo Entidad-Relación incorporando entidades débiles, atributos compuestos y multivaluados, relaciones ternarias con roles y un soporte inicial para jerarquías ISA. Estas ampliaciones no se limitaron a la parte visual, sino que afectaron

también a la persistencia, la validación, la transformación relacional y la generación de SQL.

El soporte de jerarquías ISA se planteó de forma limitada y controlada. La aplicación permite representar una generalización con una o varias especializaciones y transforma esta estructura generando una tabla para la entidad general y una tabla para cada especialización, donde la clave heredada actúa como clave primaria y clave foránea. No se incorporaron restricciones más avanzadas como totalidad o parcialidad, especializaciones solapadas o exclusivas, discriminantes o herencia múltiple.

También se revisó la generación de SQL, especialmente el tratamiento de claves foráneas y la forma de expresar el script final. El objetivo fue obtener una salida más clara y coherente con el modelo relacional generado, reduciendo el uso de sentencias `ALTER TABLE` cuando las dependencias entre tablas pueden resolverse mediante el orden de creación.

Otro resultado relevante es la mejora de la usabilidad del editor. Se reorganizó la interfaz, se revisaron los mensajes de validación y confirmación, se incorporó acceso a información de ayuda dentro de la aplicación y se añadieron operaciones como selección múltiple, ajuste de vista, exportación visual, importación JSON y generación de estructuras de ejemplo.

Durante el desarrollo también se comprobó la importancia de trabajar de forma incremental. Al tratarse de una aplicación heredada y visual, muchos cambios podían afectar a varias partes del sistema al mismo tiempo. Por ello, dividir el trabajo en tareas acotadas y combinar pruebas automáticas con revisión manual permitió avanzar con más seguridad y reducir el riesgo de introducir regresiones.

En conjunto, el resultado obtenido es una herramienta más completa que la aplicación inicial, con mayor soporte para elementos del modelo Entidad-Relación, una salida SQL más cuidada y una interfaz más clara. Su finalidad no es sustituir la explicación teórica ni competir con herramientas profesionales, sino servir como apoyo práctico en un contexto académico.

La revisión de trabajos relacionados permite situar el resultado frente a otras herramientas. En particular, ERDPlus es la alternativa más próxima, ya que también trabaja con diagramas Entidad-Relación, esquemas relacionales y generación SQL. No obstante, UBU E-R App ofrece una propuesta ajustada al alcance docente de este TFG en aspectos como la ausencia de publicidad, la posibilidad de ejecución local, el soporte de relaciones ternarias con roles, la validación orientada al usuario y la integración entre edición visual, modelo interno y generación SQL.

7.2. Líneas de trabajo futuras

A partir del trabajo realizado, pueden plantearse varias líneas de continuación. Algunas están relacionadas con ampliar el soporte del modelo Entidad-Relación, mientras que otras se orientan a reforzar el uso docente de la herramienta o a mejorar su utilidad en escenarios de mayor tamaño.

- **Ampliar el soporte de jerarquías ISA.** La versión actual incorpora un soporte inicial controlado para ISA. Como trabajo futuro se podrían añadir restricciones de totalidad o parcialidad, especializaciones solapadas o exclusivas, discriminantes y distintas estrategias de transformación relacional, como representar toda la jerarquía en una sola tabla o utilizar tablas independientes para la generalización y las especializaciones.
- **Incorporar agregaciones.** Las agregaciones quedan fuera del alcance implementado. Su incorporación requeriría revisar la representación visual, el metamodelo interno, las reglas de validación y la transformación relacional, ya que implican tratar una interrelación como participante de otra interrelación.
- **Mejorar la configuración de tipos de datos y restricciones SQL.** Actualmente, la generación SQL utiliza tipos generales suficientes para el contexto académico. Una línea futura sería permitir configurar tipos concretos, tamaños, obligatoriedad, restricciones adicionales o valores únicos por atributo.
- **Ampliar la gestión de diagramas grandes mediante vistas.** Para modelos con muchas entidades e interrelaciones, podría incorporarse un sistema de vistas que permitiera trabajar con partes del diagrama de forma separada, manteniendo un modelo global común. Esto ayudaría a mejorar la legibilidad en diagramas complejos.
- **Incorporar un banco de ejercicios y corrección automática.** La aplicación podría reforzar su dimensión docente mediante ejercicios integrados. Por ejemplo, ejercicios de diagrama E/R a modelo relacional, ejercicios inversos desde tablas o ejercicios basados en enunciados textuales. Sobre esta base, podrían añadirse mecanismos de corrección automática que comparasen la solución del alumno con una solución esperada y proporcionasen retroalimentación.
- **Permitir rúbricas configurables por el profesor.** Relacionado con la corrección automática, podría incorporarse un sistema de rúbricas

en el que el profesor definiera qué aspectos se valoran y con qué peso: entidades, atributos, claves, relaciones, cardinalidades, entidades débiles, transformación relacional o generación SQL.

- **Explorar el apoyo de inteligencia artificial en ejercicios desde enunciado.** En ejercicios basados en texto, podría estudiarse el uso de inteligencia artificial para generar propuestas iniciales, detectar diferencias con una solución de referencia o sugerir posibles errores. Esta línea debería abordarse con prudencia, manteniendo siempre la revisión docente y admitiendo que puede haber varias soluciones válidas.
- **Generar otras representaciones lógicas.** Aunque este TFG se centra en el modelo relacional y SQL, el modelo Entidad-Relación es independiente del modelo lógico final. Por ello, podría estudiarse la generación de otras representaciones, como estructuras orientadas a documentos para bases de datos tipo MongoDB.
- **Relacionar diagramas Entidad-Relación y diagramas de clases UML.** Otra posible ampliación sería generar diagramas de clases UML a partir de diagramas E/R, o realizar el camino inverso en casos controlados. Esta línea permitiría conectar la herramienta con otros modelos utilizados en ingeniería del software.
- **Ampliar la cobertura de pruebas y validaciones.** Futuras ampliaciones funcionales deberían ir acompañadas de nuevas pruebas unitarias, pruebas end-to-end y revisión manual. En particular, sería conveniente aumentar la cobertura de casos complejos de transformación relacional, importación/exportación, reconstrucción visual y generación SQL.

Bibliografía

- [1] Steven Paul Alba Alba. Ubu e-r app. <https://github.com/saa1002/draw-entity-relation>, 2026. Repositorio GitHub de la versión evolucionada del proyecto. Consultado el 30 de junio de 2026.
- [2] Biome. Biome. <https://biomejs.dev/>. [Internet; consultado 25-mayo-2026].
- [3] Peter P. Chen. The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976.
- [4] ERDPlus. Erdplus. <https://erdplus.com/>. [Internet; consultado 15-junio-2026].
- [5] ERDPlus. Erdplus terms of service. <https://erdplus.com/terms-of-service>, 2026. [Internet; consultado 30-junio-2026].
- [6] Excalidraw. Excalidraw. <https://excalidraw.com/>. [Internet; consultado 15-junio-2026].
- [7] Excalidraw. Excalidraw+ pricing. <https://plus.excalidraw.com/pricing>, 2026. [Internet; consultado 30-junio-2026].
- [8] Git. Git Documentation. <https://git-scm.com/docs>. [Internet; consultado 25-mayo-2026].
- [9] GitHub. GitHub. <https://github.com/>. [Internet; consultado 25-mayo-2026].
- [10] GitHub. Github actions documentation. <https://docs.github.com/actions>, 2026. Consultado el 30 de junio de 2026.

- [11] JGraph. mxgraph. <https://github.com/jgraph/mxgraph>. [Internet; consultado 25-mayo-2026].
- [12] JGraph Ltd. diagrams.net / draw.io. <https://app.diagrams.net/>. [Internet; consultado 15-junio-2026].
- [13] Lucid Software. Lucidchart pricing. <https://lucid.app/pricing/lucidchart>, 2026. [Internet; consultado 30-junio-2026].
- [14] Evil Martians. Lefthook. <https://lefthook.dev/>. [Internet; consultado 25-mayo-2026].
- [15] Rubén Maté Iturriaga. Draw entity-relation. <https://github.com/rubenmate/draw-entity-relation>, 2024. Repositorio GitHub del proyecto original. Consultado el 30 de junio de 2026.
- [16] Rubén Maté Iturriaga. Draw entity-relation app. Trabajo Fin de Grado, Universidad de Burgos, 2024. Memoria del proyecto previo.
- [17] Jesús Maudes Raedo. Tema 6. El modelo E/R y su representación lógica en SQL. Material docente de Bases de Datos, Grado en Ingeniería Informática, Universidad de Burgos, 2017. Material docente consultado durante la realización del trabajo.
- [18] Meta. React. <https://react.dev/>. [Internet; consultado 25-mayo-2026].
- [19] Microsoft. Playwright. <https://playwright.dev/>. [Internet; consultado 25-mayo-2026].
- [20] Microsoft. Visual studio code. <https://code.visualstudio.com/>. [Internet; consultado 25-mayo-2026].
- [21] MUI. Material UI. <https://mui.com/material-ui/>. [Internet; consultado 25-mayo-2026].
- [22] npm. npm. <https://www.npmjs.com/>. [Internet; consultado 25-mayo-2026].
- [23] OpenJS Foundation. Node.js. <https://nodejs.org/>. [Internet; consultado 25-mayo-2026].
- [24] Overleaf. Overleaf. <https://www.overleaf.com/>. [Internet; consultado 25-mayo-2026].

- [25] PostgreSQL Global Development Group. Postgresql documentation: Create table. <https://www.postgresql.org/docs/current/sql-createtable.html>, 2026. Consultado el 30 de junio de 2026.
- [26] Vercel. Vercel. <https://vercel.com/>. Accedido: 25 de junio de 2026.
- [27] Visual Paradigm. Free erd editor online. <https://online.visual-paradigm.com/diagrams/solutions/free-erd-editor-online/>, 2026. [Internet; consultado 30-junio-2026].
- [28] Visual Paradigm. Visual paradigm online pricing. <https://online.visual-paradigm.com/pricing/>, 2026. [Internet; consultado 30-junio-2026].
- [29] Vitest. Vitest. <https://vitest.dev/>. [Internet; consultado 25-mayo-2026].